

VIEWS:

In MySQL, a view is a virtual table that provides a way to simplify complex SQL queries. A view is created by defining a query and giving it a name, and then the view can be treated like a regular table in subsequent queries. This can make it easier to write and maintain queries, especially when dealing with complex joins, aggregations, or filtering.

Here is the basic syntax for creating a view in MySQL:

```
CREATE VIEW view_name AS
```

```
SELECT column1, column2, ...
```

```
FROM table1
```

```
WHERE condition;
```

Let's break down each part of the syntax:

- **CREATE VIEW view_name:** This is the beginning of the statement that creates a new view. `view_name` is the name you want to give to the view.
- **AS:** This keyword is used to define the **SELECT** statement that forms the basis of the view.
- **SELECT column1, column2, ...:** This is the **SELECT** statement that defines the columns you want to include in the view.
- **FROM table1:** This is the table you want to query to create the view.
- **WHERE condition:** This is an optional clause that specifies any conditions that the data must meet in order to be included in the view.

Once a view has been created, you can use it in subsequent queries as if it were a regular table. For example, you could write a query like this:

```
SELECT *  
FROM view_name  
WHERE column1 = 'value';
```

This query would return all rows from the view that match the condition `column1 = 'value'`.

You can also modify a view using the `ALTER VIEW` statement, or delete it using the `DROP VIEW` statement.

A view can be updated with the `CREATE OR REPLACE VIEW` statement

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

A view is deleted with the `DROP VIEW` statement.

```
DROP VIEW view_name;
```

Example:

Create a view that shows all customers from Brazil

```
CREATE VIEW [Brazil Customers] AS
SELECT CustomerName, ContactName
FROM Customers
WHERE Country = 'Brazil';
```

USER DEFINED FUNCTIONS:

In MySQL, a user-defined function (UDF) is a function that is created by the user and can be used in SQL statements just like built-in functions. A UDF can take parameters and return a value, and can be used to perform custom calculations, manipulate data, or perform other operations.

Here is the basic syntax for creating a UDF in MySQL:

```
CREATE FUNCTION function_name (parameter1 data_type, parameter2  
data_type, ...)
```

```
RETURNS return_type
```

```
BEGIN
```

```
-- function body goes here
```

```
END;
```

Let's break down each part of the syntax:

- **CREATE FUNCTION function_name:** This is the beginning of the statement that creates a new function. `function_name` is the name you want to give to the function.
- **(parameter1 data_type, parameter2 data_type, ...):** These are the input parameters for the function, along with their data types. You can specify as many parameters as you need.
- **RETURNS return_type:** This specifies the data type that the function will return.
- **BEGIN and END:** These keywords mark the beginning and end of the function's body. The function's logic should be written between these two keywords.

Within the function body, you can write any valid SQL statements, including **SELECT**, **INSERT**, **UPDATE**, and **DELETE** statements. You can also use control flow statements like **IF**, **WHILE**, and **CASE**.

Here is an example of a simple UDF that takes two numbers as input and returns their sum:

```
CREATE FUNCTION add_numbers (a INT, b INT)
RETURNS INT
BEGIN
    DECLARE result INT;
    SET result = a + b;
    RETURN result;
END;
```

In this example, the function `add_numbers` takes two integer parameters, `a` and `b`, adds them together, and returns the result.

Once a UDF has been created, you can use it in SQL statements just like any other function. For example:

```
SELECT add_numbers(2, 3); -- returns 5
```

Example to use the SQL statement inside a Function:

Write a MySQL function to calculate the total sales made by a given salesman.

```
CREATE FUNCTION total_sales(salesman_id INT) RETURNS DECIMAL(10,2)
BEGIN
    DECLARE total DECIMAL(10,2);
    SELECT SUM(amount) INTO total
    FROM Orders
    WHERE salesman_id = salesman_id;
    RETURN total;
END;
```

